Mineração de Métricas DORA em Repositórios Open-Source: Quanto Podemos Confiar nas Aproximações?

Arthur Lara Panzera[1], **Felipe Augusto Pereira**[1], **Gabriel Reis Lebron de Oliveira**[1]

[1]Curso de Engenharia de Software – PUC Minas
Belo Horizonte – MG – Brasil

arthurpanzera@outlook.com, felipeaps0918@gmail.com, reislebron@gmail.com

Abstract. *DORA metrics are the industry standard for measuring software delivery performance, but GitHub records neither production deployments nor production failures, so they can only be approximated from releases, commits and CI workflow runs. This paper mines DORA metrics from 300 popular open-source repositories that use GitHub Actions, validates the automatic criteria against human judgment, and measures how much each repository's DORA classification depends on the operational definition. The median project publishes 0.35 releases per week, has a lead time of 7.3 days and recovers from CI failures in about two hours. The two change failure rate proxies barely agree (weighted kappa of 0.02): 6.0% for CI failures and 37.5% for releases followed by a corrective release, and 28% to 45% of the repositories change their overall DORA class depending on the definition.*

Resumo. *As métricas DORA são o padrão de mercado para medir o desempenho de entrega de software, mas o GitHub não registra deploys nem falhas em produção, e elas só podem ser aproximadas a partir de releases, commits e execuções de CI. Este trabalho minera as métricas DORA de 300 repositórios open-source populares que usam GitHub Actions, valida os critérios automáticos contra o julgamento humano e mede o quanto a classificação DORA de cada repositório depende da definição operacional. O projeto mediano publica 0,35 release por semana, tem lead time de 7,3 dias e se recupera de falhas de CI em cerca de duas horas. Os dois proxies da taxa de falha quase não concordam (kappa ponderado de 0,02): 6,0% pelas falhas de CI e 37,5% pelas releases seguidas de uma release corretiva, e de 28% a 45% dos repositórios mudam de classe DORA geral conforme a definição.*

1. Introdução

As métricas do programa DORA (*DevOps Research and Assessment*), popularizadas pelo livro *Accelerate* [Forsgren et al. 2018] e revisadas anualmente pelos relatórios *Accelerate State of DevOps* [DORA 2024], tornaram-se o padrão de mercado para medir o desempenho de entrega de software. Elas avaliam a *velocidade*, pela frequência de deploy e pelo tempo entre um commit e sua entrega em produção (*lead time for changes*), e a *estabilidade*, pela proporção de deploys que causam falha (*change failure rate*, CFR) e pelo tempo para se recuperar delas [DORA a]. Em 2024, o DORA acrescentou a *deployment rework rate* [DORA b].

Repositórios open-source permitem medi-las em larga escala, mas o GitHub não registra deploys nem falhas em produção, só releases, commits e execuções de integração

contínua (CI). O cálculo depende, portanto, de *proxies*: uma release publicada é tomada como deploy, e uma execução de CI que falhou, como falha. Esses proxies podem enganar: uma biblioteca que publica uma release não coloca nada em produção; quem o faz são seus usuários.

Este trabalho minera as métricas DORA de repositórios open-source com 1.000 estrelas ou mais que usam GitHub Actions, em uma janela de 12 meses (1º de outubro de 2025 a 30 de setembro de 2026), e questiona a confiabilidade dos valores obtidos: os critérios automáticos são comparados com o julgamento de três avaliadores sobre uma amostra de 60 repositórios, e uma análise de sensibilidade mede o quanto a classificação DORA muda quando a definição operacional muda. O estudo responde a sete questões de pesquisa, cada uma com uma hipótese informal escrita **antes** da coleta, para que o confronto com os resultados (Seção 4) não seja influenciado por eles:

RQ1. Qual a frequência de deploy? *H1:* mediana entre uma release por mês e uma por semana (*Medium*), com poucos *Elite*, pois projetos open-source agrupam mudanças em versões planejadas.

RQ2. Qual o tempo entre um commit e seu deploy? *H2:* semanas, com a maioria *Medium* ou *Low*; a variante por release (a), baseada no commit mais antigo, seria sistematicamente maior que a por commit (b), pois um único commit antigo "esquecido" a eleva muito.

RQ3. Qual a taxa de falha das mudanças? *H3:* CFR de CI (a) entre 10% e 25%; CFR de entrega (b) abaixo de 15%; baixa concordância entre as duas, que medem falha de pipeline e defeito entregue.

RQ4. Qual o tempo de recuperação após uma falha de CI? *H4:* entre uma hora e um dia (*High*), com cauda longa e menos de 10% de episódios censurados.

RQ5. Mais deploys implicam mais ou menos falhas? *H5:* correlação de Spearman fraca ($|\rho| < 0{,}3$) nas duas variantes, como afirma o DORA; levemente positiva no CFR (b), por artefato da definição.

RQ6. Que características se associam a melhor desempenho? *H6:* tipo de projeto e número de contribuidores seriam os principais fatores; popularidade e linguagem, desprezíveis após a correção de Holm.

RQ7. O quanto a classificação depende da definição operacional? *H7:* ao menos 30% dos repositórios mudariam de classe, com kappa ponderado entre 0,41 e 0,60, sobretudo ao trocar a unidade de deploy.

Como questão bônus (RQ8), comparamos a *deployment rework rate* com o CFR de entrega. Como a opção do bônus foi escolhida depois da coleta, ela não tem hipótese prévia. As contribuições são: (i) um pipeline reprodutível de mineração das métricas DORA; (ii) um conjunto de dados de 300 repositórios, com dicionário de dados; (iii) a validação dos critérios automáticos contra rótulos humanos; (iv) uma análise de quão robustas são as conclusões frente à definição operacional; (v) a classificação automática do tipo de projeto, validada contra os rótulos humanos, que permite usar os 300 repositórios, e não só os 60 rotulados, na RQ6; e (vi) a separação das falhas intermitentes (*flaky*) no CFR de CI, que mede o quanto elas pesam no proxy de falha.

2. Metodologia

2.1. Coleta e seleção dos repositórios

Um único pipeline, disponível no repositório do grupo, coleta os dados das APIs REST e GraphQL do GitHub por um cliente HTTP próprio, sem bibliotecas prontas de acesso à API, e grava dados brutos, métricas e validação em CSVs descritos em um dicionário de dados. Cada resposta fica em um cache local (SQLite), o que permite retomar uma coleta interrompida; o cliente respeita o *rate limit* pelos cabeçalhos `X-RateLimit-*` e repete erros temporários com espera exponencial. Releases vêm da API REST; tags e comparações entre releases, da GraphQL, refeitas pelo `compare` da REST, com paginação, quando passam de 1.000 commits. Os *workflow runs* do *default branch* disparados por `push` foram consultados mês a mês, e cada mês que atingia o teto de 1.000 resultados foi dividido ao meio até caber; nenhuma das 3.872 consultas ficou truncada, e foram obtidos 528.304 runs.

Os candidatos são todos os repositórios com 1.000 estrelas ou mais, exceto *forks* e arquivados. Como a busca do GitHub devolve no máximo 1.000 resultados por consulta, ela foi fatiada em cinco faixas de estrelas, divididas ao meio recursivamente até caberem no teto, o que trouxe a lista completa de 60.043 candidatos, congelada em arquivo. Os candidatos foram avaliados em ordem aleatória e reproduzível (*hash* SHA-256 do nome com semente fixa), e a amostra reúne os primeiros 300 que passaram nos critérios, o que equivale a sortear 300 entre os elegíveis (Tabela 1). Ela tem mediana de 3.310 estrelas, com predomínio de TypeScript (76 repositórios), Go (40), Rust (38) e Python (35).

Tabela 1. Funil de seleção dos repositórios.

Etapa	Restantes	Descartados
Candidatos (1.000 estrelas ou mais)	60.043	–
Avaliados até completar a amostra	2.160	57.883
Usam GitHub Actions	1.238	922
Pelo menos 5 releases na janela	411	827
Pelo menos 50 runs válidos na janela	300	111

2.2. Definições operacionais

Seguimos as definições comuns da disciplina. O deploy é uma release publicada (`draft` falso e sem pré-releases); pré-releases e tags são variantes da RQ7. A data de um commit é `commit.author.date`, e a release anterior é a publicada imediatamente antes. Um run é sucesso com `conclusion` igual a `success` e falha com `failure`, `timed_out` ou `startup_failure`; os demais valores são ignorados.

Frequência (RQ1): releases na janela por semana. **Lead time (RQ2):** para cada release, os commits da comparação com a anterior; na variante (a), publicação menos o commit mais antigo, com mediana entre as releases; na (b), publicação menos cada commit, com mediana de todos os commits. Ficam de fora a primeira release da história, as comparações com erro (ex.: sem ancestral comum) e os commits com data posterior à release. **CFR (RQ3):** na variante (a), runs com falha sobre falhas mais sucessos; na (b), proporção de releases seguidas, em até sete dias, por uma release corretiva, excluindo do denominador as publicadas nos últimos sete dias da janela (censura). Como análise

de sensibilidade do CFR (a), retiramos as falhas seguidas de sucesso no mesmo *workflow* e commit (intermitentes) e contamos os runs reexecutados que passaram, cuja falha original a API não lista. **Tempo de recuperação (RQ4):** por *workflow*, um episódio vai da primeira falha após um sucesso (`run_started_at`) ao fim do próximo sucesso (`updated_at`); o valor do repositório é a mediana dos episódios, e os que não terminam na janela são registrados como censurados.

Classificação DORA. Cada métrica é classificada em *Elite*, *High*, *Medium* ou *Low* pelos cortes da disciplina; a classe geral é a mediana das notas (4 a 1), arredondada para baixo, e usa as três métricas restantes quando não há tempo de recuperação. A combinação de referência (C1) usa release, lead time (a) e CFR (a). Na RQ7, comparamos C1 com C2 (release e pré-release, lead time (b), CFR (b)) e C3 (tag, lead time (b), CFR (a)), pela proporção de repositórios que mudam de classe e pelo kappa de Cohen ponderado linearmente. Na RQ5 usamos a correlação de Spearman; na RQ6, o teste de Kruskal-Wallis com correção de Holm, ε^2 e Cliff's delta.

2.3. Validação manual

Sorteamos, com semente fixa, 60 dos 300 repositórios e cinco releases de cada (300 releases). Cada integrante rotulou a amostra sozinho, sem consultar os demais e sem ver a saída das heurísticas (um deles com auxílio de IA, autorizado pelo professor; ver Seção 5), em planilhas geradas pelo pipeline. Foram rotulados o tipo do projeto (biblioteca/framework, aplicação/serviço, ferramenta CLI ou outro), se as releases são entregas reais ao usuário e se cada release é corretiva. A concordância foi medida pelo kappa de Fleiss, interpretado pela escala de Landis e Koch [Landis and Koch 1977] (Tabela 2). O consenso é o rótulo unânime ou, sem unanimidade, o da maioria; casos 1-1-1 são decididos em reunião. O kappa de *entregas reais* é baixo apesar de 94% de concordância porque quase todos receberam "sim", o que eleva a concordância esperada por acaso; houve um único caso sem maioria, nessa dimensão.

Tabela 2. Concordância entre os três avaliadores na amostra-ouro.

Dimensão	Itens	Concordância	Kappa	Interpretação
Tipo do projeto	60	92,2%	0,88	quase perfeita
Entregas reais	60	93,9%	0,29	razoável
Release corretiva	300	84,9%	0,69	substancial

A heurística de release corretiva foi avaliada contra o consenso por precisão, recall e F1 e refinada a partir dos erros de cada versão até superar F1 de 0,70 (Tabela 3). A v1 marcava como corretivas muitas releases *minor* e *major* só por terem algum commit de correção; a v2, ao exigir os dois sinais, perdeu recall em projetos com várias linhas de versão, em que a release publicada logo antes é de outra linha. Por isso a v3, adotada, olha apenas a própria versão: é corretiva a release *patch* (X.Y.Z com $Z > 0$).

O tipo do projeto foi rotulado por humanos só na amostra-ouro. Para os outros 240 repositórios, ele foi atribuído por uma classificação automática baseada em palavras-chave da descrição, dos tópicos e do início do README; a primeira versão acertou 73% dos 60 repositórios rotulados, e a segunda, refinada pelos erros, 95% (F1 macro de 0,93). Na amostra-ouro, prevalece o rótulo humano.

Tabela 3. Versões da heurística de release corretiva frente ao consenso (300 releases, 126 corretivas). Commit de correção é aquele cujo título contém *fix*, *hotfix*, *bugfix* ou *revert*.

Versão	Regra	Precisão	Recall	F1
v1	Só o *patch* muda em relação à anterior OU há commit de correção	0,47	0,96	0,63
v2	Só o *patch* muda em relação à anterior E há commit de correção	0,68	0,61	0,64
v3	A própria versão é um *patch* (X.Y.Z com $Z > 0$)	0,58	0,94	0,72
v4	A própria versão é um *patch* E há commit de correção	0,67	0,76	0,71

3. Resultados

Os valores são calculados por repositório e resumidos por mediana e intervalo interquartil (IQR), pois as distribuições são assimétricas (Tabelas 4 e 5).

Tabela 4. Métricas DORA dos 300 repositórios (mediana e IQR).

RQ	Métrica	n	Mediana	IQR (Q1 – Q3)	Classe da mediana
RQ1	Deploys por semana	300	0,35	0,19 – 0,85	Medium
RQ2 (a)	Lead time por release (dias)	300	7,3	1,0 – 24,8	Medium
RQ2 (b)	Lead time por commit (dias)	300	5,7	0,9 – 25,7	High
RQ3 (a)	CFR de CI	300	5,98%	2,55% – 12,84%	Elite
RQ3 (b)	CFR de entrega	299	37,5%	16,7% – 69,6%	Medium
RQ4	Tempo de recuperação (horas)	283	1,97	0,55 – 12,21	High

RQ1. A mediana é de 0,35 release por semana (cerca de 1,5 por mês). Só 20,7% dos repositórios publicam ao menos uma por semana, e 5 chegam a Elite; a média (0,99) é quase o triplo da mediana, puxada por poucos projetos que publicam a cada *merge* (máximo de 23,6 por semana).

RQ2. O lead time (a) tem mediana de 7,3 dias, e o (b), de 5,7. A variante (b) é menor em 193 repositórios, mas maior em 105, com correlação $\rho = 0{,}72$ entre as duas. Foram descartados 448 commits com data posterior à release.

RQ3. O CFR de CI tem mediana de 5,98%, e 79,7% dos repositórios são Elite nessa dimensão; o CFR de entrega tem mediana de 37,5% (Medium), com 429 releases (2,8%) censuradas. Retirar as falhas intermitentes detectáveis (13 de 44.545) não altera o CFR de CI; contando uma falha oculta para cada um dos 2.021 runs reexecutados que passaram, a mediana sobe só para 6,61%.

RQ4. A mediana é de 1,97 hora (High), com cauda longa (média de 25 horas). Houve 12.911 episódios recuperados e 106 censurados (0,8%). Dezessete repositórios não têm tempo de recuperação: 11 sem falha de CI e 6 com falhas só antes do primeiro sucesso.

Na classe geral, 43,3% dos repositórios são High e 37,7% Medium. A frequência de deploy é a dimensão que mais a limita: fica abaixo da classe geral em 166 repositórios, contra 70 no lead time, 13 no CFR e 11 na recuperação. A regra de arredondamento também pesa: 142 repositórios têm mediana das notas terminada em 0,5, e arredondá-la para cima elevaria os Elite de 34 para 89.

Tabela 5. Classificação DORA na combinação de referência (C1), em número de repositórios.

Dimensão	Elite	High	Medium	Low	Sem dados
Frequência de deploy	5	57	140	98	0
Lead time (a)	73	76	87	64	0
CFR (a)	239	32	13	16	0
Tempo de recuperação	102	135	38	8	17
Geral	**34**	**130**	**113**	**23**	**0**

RQ5. Com o CFR de CI, a correlação com a frequência é fraca e não significativa (ρ = 0,10, p = 0,072, n = 300); com o CFR de entrega, é forte e positiva (ρ = 0,63, p $<$ 0,001, n = 299) (Figura 1).

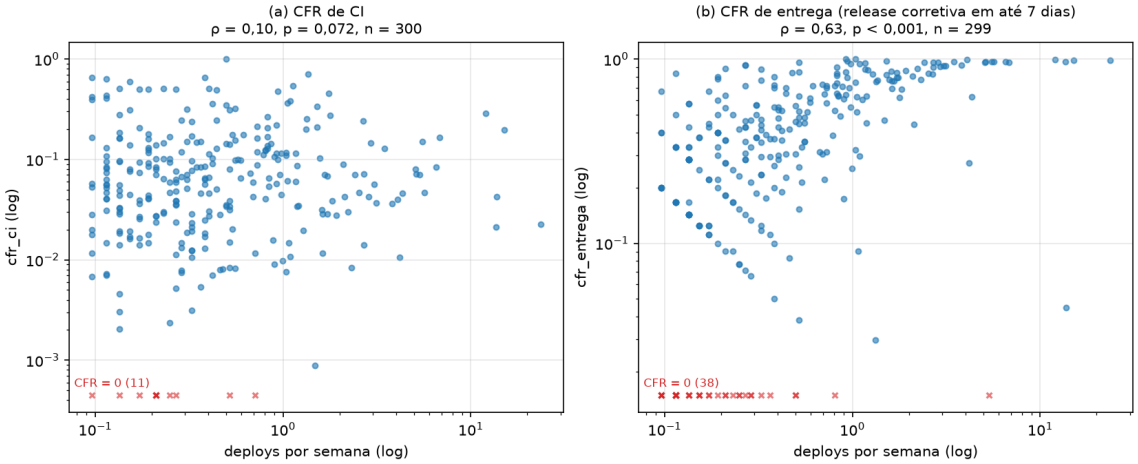


Figura 1. Frequência de deploy contra CFR de CI (a) e CFR de entrega (b), em escala logarítmica. Os pontos marcados com "x" têm CFR igual a zero.

RQ6. Comparamos as quatro métricas da C1 entre grupos de cinco fatores: linguagem, quartis de estrelas, de contribuidores e de idade, e tipo de projeto. Dos 20 testes, só 3 foram significativos após Holm, todos de efeito moderado (Tabela 6). Os repositórios mais novos (Q1, até 1,5 ano) publicam com mais frequência e têm lead time bem menor que os mais antigos (Q4, acima de 10,3 anos), e o lead time cresce com o número de contribuidores (de até 27, no Q1, a mais de 178, no Q4). Linguagem, popularidade e tipo de projeto não tiveram efeito em nenhuma métrica, e nenhum fator afetou o CFR de CI ou a recuperação.

Tabela 6. Testes significativos da RQ6 após a correção de Holm: mediana da métrica em cada quartil (Q1 é o grupo de menor valor) e Cliff's delta entre Q4 e Q1.

Fator	Métrica	Q1	Q2	Q3	Q4	p (Holm)	ε^2	δ (Q4 $\times$ Q1)
Idade	Deploys/semana	0,61	0,31	0,33	0,27	0,002	0,071	$-$0,45 (médio)
Idade	Lead time (h)	36	342	220	264	$<$ 0,001	0,120	0,47 (médio)
Contribuidores	Lead time (h)	75	133	186	340	0,018	0,055	0,37 (médio)

RQ7. A classe geral muda para 45,3% dos repositórios entre C1 e C2 (kappa de 0,349) e para 28,0% entre C1 e C3 (kappa de 0,655) (Tabela 7). De C1 para C2, 102

repositórios descem e 34 sobem, e os Elite caem de 34 para 7; em C3, com tags, 63 sobem e 21 descem, e os Elite vão a 49. A variante de CFR é a que mais altera a classificação (76,6% mudam, kappa de 0,024), seguida da de lead time (46% a 49%); a unidade de deploy muda a classe de frequência de só 9% a 18% dos repositórios.

Tabela 7. Proporção de repositórios que mudam de classe e kappa de Cohen ponderado entre as combinações (C1: release, lead time (a), CFR (a); C2: release e pré-release, (b), (b); C3: tag, (b), (a)).

	C1 × C2		C1 × C3		C2 × C3	
Dimensão	**Mudam**	**Kappa**	**Mudam**	**Kappa**	**Mudam**	**Kappa**
Frequência	10,3%	0,852	18,3%	0,744	9,0%	0,883
Lead time	46,0%	0,527	49,0%	0,488	8,0%	0,927
CFR	76,6%	0,024	0,0%	1,000	76,6%	0,024
Geral	**45,3%**	**0,349**	**28,0%**	**0,655**	**43,0%**	**0,448**

RQ8 (bônus). Escolhemos a *rework rate*, proporção de releases corretivas sobre o total, com a mesma heurística do CFR (b). A alternativa, evolução por trimestre com Mann-Kendall, foi descartada porque a janela tem só quatro trimestres, e com n = 4 o menor p-valor possível é 0,089. A *rework rate* tem mediana de 68,0% (IQR de 42,9% a 86,7%) e correlação moderada com o CFR (b) (ρ = 0,50, p $<$ 0,001), mas é sistematicamente maior (Wilcoxon pareado, p $<$ 0,001): supera o CFR (b) em 203 repositórios, empata em 23 e fica abaixo em 73.

4. Discussão

A Tabela 8 confronta as hipóteses com os resultados.

Tabela 8. Hipóteses informais *versus* resultados.

RQ	Hipótese	Resultado	Situação
RQ1	Entre 1/mês e 1/semana (Medium), poucos Elite	0,35/semana (Medium); 5 Elite	Confirmada
RQ2	Semanas; (a) sistematicamente maior que (b)	7,3 dias (a) e 5,7 dias (b); (b) menor em 64% dos repositórios	Parcial
RQ3	CFR (a) de 10% a 25%; CFR (b) abaixo de 15%; baixa concordância	CFR (a) 6,0%; CFR (b) 37,5%; kappa 0,024	Parcial
RQ4	Entre 1 hora e 1 dia; menos de 10% censurados	1,97 hora; 0,8% censurados	Confirmada
RQ5	$\|\rho\| < 0{,}3$ nas duas variantes	ρ = 0,10 (a); ρ = 0,63 (b)	Parcial
RQ6	Tipo e contribuidores principais; popularidade e linguagem sem efeito	Idade é o principal; tipo sem efeito; popularidade e linguagem sem efeito	Parcial
RQ7	Ao menos 30% mudam; kappa de 0,41 a 0,60; maior efeito da unidade de deploy	28% a 45% mudam; kappa de 0,35 a 0,66; maior efeito do CFR	Parcial

Velocidade. As hipóteses se confirmaram em grande parte: os projetos publicam cerca de uma release a cada três semanas, e um commit leva da ordem de uma semana

para ser entregue, coerente com a publicação em lotes. A surpresa veio do lead time: a variante (a) é menor que a (b) em um terço dos repositórios, porque a (b) dá mais peso às releases com muitos commits, que acumulam commits antigos. A escolha entre as duas altera a classe de quase metade dos repositórios.

Estabilidade. As hipóteses falharam em direções opostas. O CFR de CI ficou abaixo do esperado, provavelmente porque as mudanças já são testadas nos *pull requests*. O CFR de entrega ficou muito acima: em parte porque muitos projetos publicam um *patch* poucos dias após cada versão, em parte pela heurística, que marcou como corretivas 68% das releases rotuladas, contra 42% do julgamento humano. A concordância quase nula entre as variantes confirma que elas medem fenômenos distintos, nenhum deles a falha em produção.

Velocidade e estabilidade (RQ5). Com o CFR de CI, os dados apoiam a afirmação do DORA de que velocidade e estabilidade não são um *trade-off*. A correlação forte com o CFR de entrega é, como antecipado, em boa parte artefato da definição: quanto mais releases, mais chances de um *patch* sair em até sete dias depois de outra. A conclusão sobre o *trade-off* depende, portanto, da variante.

Fatores (RQ6). A hipótese se confirmou só em parte: popularidade e linguagem ficaram sem efeito, como esperado, mas o tipo de projeto também, e o principal fator foi a idade. Projetos com menos de 1,5 ano publicam com o dobro da frequência e têm lead time sete vezes menor que os mais antigos, possivelmente por estarem em fase de iteração rápida, enquanto projetos maduros agrupam mudanças em versões planejadas. Mais contribuidores se associaram a lead time maior, e não a recuperação mais rápida, o que é coerente com processos de revisão mais longos em projetos grandes.

Sensibilidade (RQ7) e *rework rate* (RQ8). A classificação depende da definição, como esperado, mas pesou mais a variante de CFR que a unidade de deploy: a distribuição geral se mantém, mas a classe de cada repositório muda, e só as conclusões sobre a frequência de deploy são robustas. A *rework rate* conta as releases de correção, e o CFR (b), as que precisaram de correção em até sete dias; por isso projetos que publicam *patches* espaçados têm *rework rate* alta e CFR (b) nulo.

5. Ameaças à Validade

Seguimos as quatro categorias de [Wohlin et al. 2012].

Construto. Release não é deploy, e falha de CI não é falha em produção. Os avaliadores consideraram as releases entregas reais em 57 dos 60 repositórios, mas com kappa de só 0,288, o que indica um critério ambíguo nos casos de fronteira. Falhas intermitentes pesam pouco no CFR de CI (cerca de 4% das falhas, mesmo contando as reexecuções ocultas). A heurística de release corretiva tem F1 de 0,715, com precisão de 0,578 e recall de 0,937, e superestima as correções (68% contra 42%), de modo que o CFR (b) e a *rework rate* são limites superiores. Ela e a classificação do tipo de projeto (F1 macro de 0,932) foram ajustadas e avaliadas nos mesmos dados rotulados, o que torna seus F1 otimistas; erros de classificação do tipo tendem a atenuar diferenças na RQ6. A data de autor dos commits é distorcida por *rebase* e *cherry-pick*, e os cortes da classificação são uma adaptação fixa das faixas do DORA, sensível à regra de arredondamento.

Interna. Os fatores da RQ6 foram testados um a um, e idade e número de con-

tribuidores são correlacionados, de modo que o efeito da idade não pode ser isolado. Os critérios de inclusão eliminaram 827 dos 1.238 repositórios com Actions por falta de releases, favorecendo projetos que publicam com regularidade. A censura foi baixa (0,8% dos episódios e 2,8% das releases). Um dos avaliadores rotulou com auxílio de IA, o que pode reduzir a independência entre os rótulos, efeito limitado pelo consenso por maioria. A coleta, de 7 a 13 de outubro de 2026, pode diferir de uma replicação posterior.

Externa. A amostra foi sorteada entre todos os repositórios com 1.000 estrelas ou mais que usam Actions e publicam releases, mas os resultados não se estendem a projetos menos populares, a projetos que entregam sem releases ou a equipes corporativas, alvo original do DORA, e refletem uma única janela de 12 meses.

Conclusão. Os 20 testes da RQ6 foram corrigidos por Holm, e os três significativos têm efeito moderado; alguns subgrupos são pequenos (10 repositórios do tipo "outro"). A correlação da RQ5 com o CFR (b) é em parte induzida pela definição e não indica causalidade. F1 e kappa foram estimados em 60 repositórios e 300 releases, sem intervalo de confiança, de modo que diferenças pequenas, como entre as versões v3 e v4 da heurística, não são distinguíveis.

Disponibilidade

Repositório do grupo: `https://github.com/Lab-Experimentao/Lab03-MineracaoMetricasDora`.
GitHub Projects do grupo: `https://github.com/orgs/Lab-Experimentao/projects/2/views/2`.

Referências

DORA. DORA's software delivery metrics: the four keys. `https://dora.dev/guides/dora-metrics-four-keys/`. Acesso em: out. 2026.

DORA. A history of DORA's software delivery metrics. `https://dora.dev/insights/dora-metrics-history/`. Acesso em: out. 2026.

DORA (2024). Accelerate State of DevOps Report. `https://dora.dev/research/`. Acesso em: out. 2026.

Forsgren, N., Humble, J., and Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution.

Landis, J. R. and Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer.